

FEBAUTO Racing

Member guide, release 2

Part I The box: your ROS 2 environment

Part II Drivers: write, score, submit

Part III The car

Formula Electric at Berkeley

September 2026, platform release v0.1.0

https://github.com/FEBAutonomous/feb-racing

https://febautonomous.github.io/feb-racing/

Contents

1 What changed, in one minute 2

I The box 2

2 Setup 2

3 How it fits together 4

4 The commands 4

II Drivers 5

5 A driver is one package 5

6 The contract 6

7 The dead-man, and the supervisor 6

8 Score and submit 7

III The car 7

9 From a merged driver to the wheels 7

10 When something is wrong 8

A Every track 9

Verified end to end, every command as written, on Linux (Ubuntu 22.04) and macOS (Apple Silicon) on 2026-10-21, and on Windows 11 (PowerShell, Docker Desktop) on 2026-10-24.

1 What changed, in one minute

Release 2 replaces the throwaway devkit container with **the box**: a Linux machine with ROS 2 Humble inside Docker that keeps what you install, sees your whole FEB folder, and runs the same on the car's Jetson. Your driver talks the real car's topics, so what laps in the simulator drives the car unchanged. The launcher is `feb`; `feb-sim` is gone.

Was	Is
`./feb-sim run`	`./feb up`
`./feb-sim shell`	`./feb shell`, or `./feb code` for VS Code inside the box
container wiped on every run	the box persists; `./feb reset` wipes it on purpose
only `stack/` visible inside	all of `~/FEB` visible inside
topics `/autodrive/roboracer_1/*`, throttle in $[0, 1]$	`/scan`, `/odom`, `/imu`, camera in; `/drive` (m/s, rad) out
one practice track	every practice track of your driver's kind
submit posts a result	submit posts the results and your code

Part I
The box

2 Setup

1. Install, once

	Install
Mac	Docker Desktop (or OrbStack). VS Code, then its Extensions view, search *Dev Containers*, Install. Foxglove from `https://foxglove.dev/download`.
Windows	In PowerShell `wsl --install`, then `wsl --update`, reboot: that is Docker's engine, you never open Ubuntu. Then Docker Desktop. Python 3 from `https://python.org` with *Add python.exe to PATH* ticked. Git for Windows from `https://git-scm.com`. VS Code with the Dev Containers extension (Extensions view, search, Install). Foxglove.
Linux	`sudo apt install docker.io`, `sudo usermod -aG docker $USER`, log out and in. VS Code with Dev Containers. Foxglove.

Sign-ins: Docker Desktop offers one, skip it. Foxglove needs a free account the first time it opens. VS Code needs none.

And one GitHub thing: an SSH key on your account. If the clone below stops with `Permission denied (publickey)`, this machine has no key on your GitHub account yet. Either `ssh-keygen -t ed25519`, Enter three times, then paste the contents of `~/.ssh/id_ed25519.pub` into GitHub, Settings, SSH and GPG keys, New SSH key, and clone again. Or clone over https, `https://github.com/FEBAutonom` and add the key before your first `feb submit`. On Windows, `ssh-keygen` runs in PowerShell too, and the key is in `C:\Users\you\.ssh`.

Where things live: Mac, under `/Users`, which `~/FEB` already is. Windows: every command in this guide is typed in **PowerShell**, the same `./feb` commands, and `~/FEB` is `C:\Users\you\FEB`.

The box is Linux inside Docker; the simulator is the native Windows build on your own GPU. Foxglove is optional everywhere; the browser desktop covers most of it.

2. Had the old devkit? Delete it

Release 2 rewrote the repository, so the old clone cannot be updated. Delete it; skip this if you never had it.

```
rm -rf ~/feb-racing            # or wherever it is;  Windows: rm -r -fo ~/feb-racing
```

Wrote code in its `stack/` folder? Copy that folder somewhere safe before deleting, and drop it into the new `stack/` after step 3. It runs again once it has a card (section 5).

3. Clone and set up

```
mkdir -p ~/FEB && cd ~/FEB
git clone git@github.com:FEBAutonomous/feb-racing.git && cd feb-racing
./feb setup --name "Your Name"
```

Windows, PowerShell, one line at a time (no `&&` there):

```
mkdir ~/FEB; cd ~/FEB
git clone git@github.com:FEBAutonomous/feb-racing.git; cd feb-racing
./feb setup --name "Your Name"
```

`setup` pulls the image (a few GB, once), downloads the simulator app for your OS, and creates your box. About twenty minutes, nearly all of it download. Your name is how results appear on the scoreboard; ask the lead to add it to `submissions.yaml` with your GitHub login.

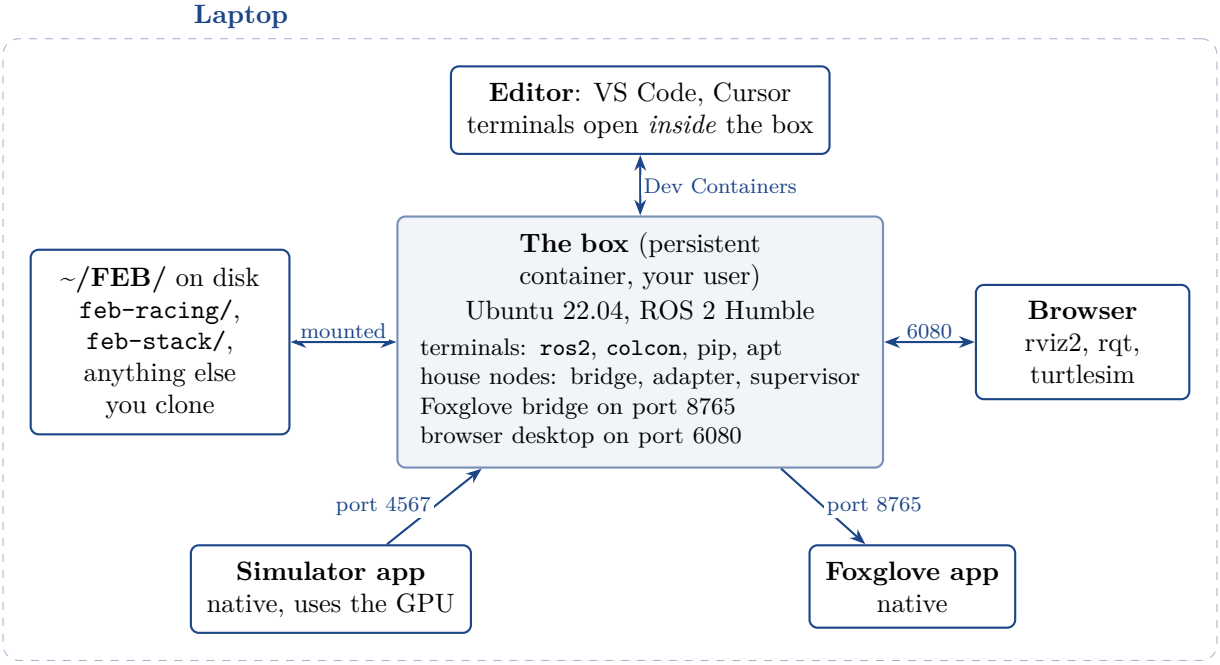
4. First drive

```
./feb up
```

The simulator opens, and Foxglove with it if it is installed. Arrow keys drive. Close the simulator window, or Ctrl-C in the terminal, to stop. Closing the *terminal* instead leaves the run going; `./feb stop` ends it. `./feb up --driver feb_driver` races the starter driver instead. Mac, first time only: if macOS refuses to open the simulator, right-click `FEB Simulator.app` in `~/.feb-sim/app/mac` (Finder: Cmd-Shift-G, paste the path) and choose Open.

That was the one and only clone. Every later release arrives with `./feb update`.

3 How it fits together



The box is one container, made once by `setup`, started by whichever command needs it, kept until `./feb reset`. Your `~/FEB` folder is mounted into it at `/home/feb/FEB`: the same files, not a copy, so edit on the laptop with anything or inside the box, it makes no difference. What you `apt` or `pip` install lives in the box and goes with a reset or a new image; your files never do.

4 The commands

Command	What it does
`./feb shell`	a terminal in the box, in the folder you ran it from; open as many as you like
`./feb shell --gui`	the same, plus the box's desktop in a browser tab (rviz2, rqt, turtlesim)
`./feb code`	VS Code attached to the box (`--cursor` for Cursor)
`./feb up`	house nodes, simulator, Foxglove; drive by hand; Ctrl-C ends it
`./feb new NAME`	your own driver in `stack/`, the starter copied and renamed; `--from feb_cone_driver` for the cone template, `--ws DIR` for another folder
`./feb up --driver NAME`	your driver races; `--track T`, `--mode-name M`, `--noise 1`, `--opponent normal`
`./feb up --deadman --driver NAME`	the car's rules in the simulator (section 7)
`./feb stop`	ends `feb up` from another terminal; the box stays
`./feb logs -f`	what `feb up` printed
`./feb practice --driver NAME`	scored runs on every practice track of the driver's kind; the first run pulls the league image, a few GB
`./feb submit`	results to the scoreboard, code to the team repo, one pull request each
`./feb update`	git pull, newer image, newer simulator; a new image removes the box, and the next command makes a fresh one
`./feb reset`	removes the box; the next command makes a fresh one in half a minute; your files are untouched

Inside the box there is nothing of ours to learn: `ros2`, `colcon`, `pip`, `ros2 bag record -a`, and `rviz2 &` which lands on the browser desktop.

rviz in the browser. With `./feb up` running in one terminal, in a second one: `./feb shell --gui`: a browser tab opens on the box's desktop, a plain Ubuntu logo for now. In that terminal `rviz2 &`, and rviz appears on the desktop. Fixed Frame `odom`; Add, By topic, `/scan` (LaserScan, style Points, size 4) and `/odom`. A topic that stays empty wants Reliability Policy set to Best Effort. **Foxglove app:** Open connection, Foxglove WebSocket, `ws://localhost:8765`; the bridge runs whenever `feb up` does.

Part II
Drivers

5 A driver is one package

One command copies the template into `stack/` under your name and renames it inside and out (a ROS package carries its name in four places); pick a name that is yours, it is what the scoreboard and the car will call the driver:

```
./feb new my_driver                # --from feb_cone_driver for the cone template, --ws
    DIR for another folder
./feb up --driver my_driver        # builds and races it
```

It refuses to touch a folder that already exists, so it cannot nest copies; a broken copy from before is deleted first (`rm -rf stack/my_driver`). Same command on every OS, in a laptop terminal.

File	Holds
`my_driver/reactive_driver.py`	the node: scan in, speed and steering out
`config/driver.yaml`	every tuning number, one per line with a comment
`config/driver.car.yaml`	only the numbers that differ on the real car; loaded on top, on the car only
`launch/drive.launch.py`	starts the node with the yaml; includes `feb_mission` if present
`card.yaml`	the driver's card (below)

```
name: my_driver
kind: walls                # walls | cones | any
maintainer: your name
packages: [my_driver]      # what submit copies: your own packages
modes:
  trackdrive: {launch: drive.launch.py, max_speed: 3.0}
  inspection: {launch: drive.launch.py, max_speed: 0.8}
default_mode: trackdrive
```

A mode is a launch file plus a speed cap; `inspection` is the slow first lap of anything new, `trackdrive` the race. Edit the Python, save, run again: no rebuild for Python. Keep the package in git, `stack/` is ignored by this repo on purpose.

A project with a number: localisation

`./feb new my_localizer --from feb_localizer` gives you a driver that already drives, maps and remembers, built around one file, `localizer.py`: a class with `predict` (dead reckoning, shipped) and two empty methods, `update_mapping` and `update_racing`. Lap one is reactive while the grid is built at your pose; lap two follows the recorded path on your pose, and with dead reckoning alone it drives into the walls on `porto`. `ros2 run my_localizer score bags/porto_laps` (in

`./feb shell`) replays a recorded lap with ground truth through your class and prints the drift, no simulator needed. Make that number go down. The package README has the rest.

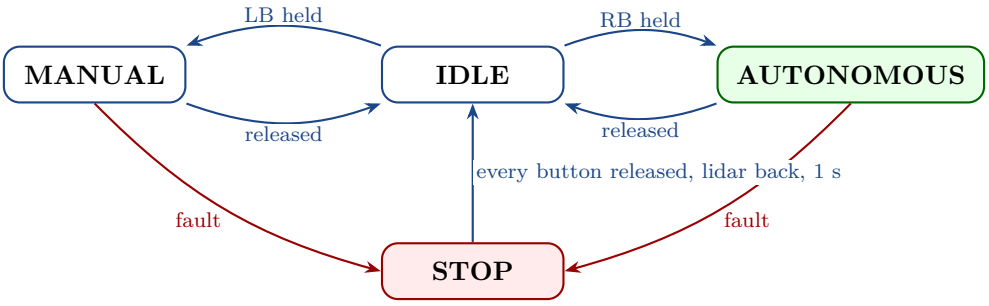
6 The contract

Topic	Type	
`/scan`	LaserScan	the lidar, 1080 beams over 270°, frame `laser`
`/odom`	Odometry	pose and speed, dead-reckoned from the wheels and the gyro
`/imu`	Imu	
`/camera/color/image_raw`	Image	the front camera (cone tracks)
`/drive`	AckermannDriveStamped	**yours**: `drive.speed` in m/s, `drive.steering_angle` in rad, at most ±0.5236
`/feb/state`	String	what the supervisor is doing: IDLE, MANUAL, AUTONOMOUS, STOP

These are the RoboRacer car's topics. In the simulator an adapter provides them; on the car the VESC and lidar drivers do. A command older than half a second stops the car. The simulator's own `/autodrive/...` topics remain for debugging; the car does not have them.

7 The dead-man, and the supervisor

On the car nothing moves unless a button on the gamepad is held. The supervisor is the node that enforces it; it sits between every driver and the wheels, in the simulator and on the car alike.



fault: lidar silent for 0.5 s, the X button, or a collision imminent

- **IDLE**: the car sits. Your stack is running and publishing `/drive`; it goes nowhere.
- **MANUAL**, LB held: the sticks drive. For positioning the car and checking wiring.
- **AUTONOMOUS**, RB held: your `/drive` reaches the wheels, clipped to the mode's cap.
- **STOP**: brakes. Entered when the lidar goes silent, the stop button is pressed, or a collision is imminent; left for IDLE only once every button is released and the sensors are back.
- Release either button: zero speed, which is a brake, within a third of a second.

In the simulator, `./feb up --deadman --driver NAME` runs the same rules with the keyboard as the pad: hold SPACE for RB, hold M for LB with the arrows driving, X for the stop button, Q to quit. The keys go to the *terminal* you ran it in, so keep that window focused, not the simulator's. The nodes' output goes to `./feb logs -f` in another terminal. Run every driver this way once before it goes near the car.

Your stack and the state. The button never starts or stops your stack; it only gates its output. So anything that remembers, an MPC's warm start, a localiser, an integrator, is stale when RB goes down. The rule: *on AUTONOMOUS, start clean.* `feb_mission`, a forty-line package in

the starter kit, turns `/feb/state` into two hooks, `on_go` and `on_stop`, and a live flag; a stack with memory fills them in. The gap follower needs none of it.

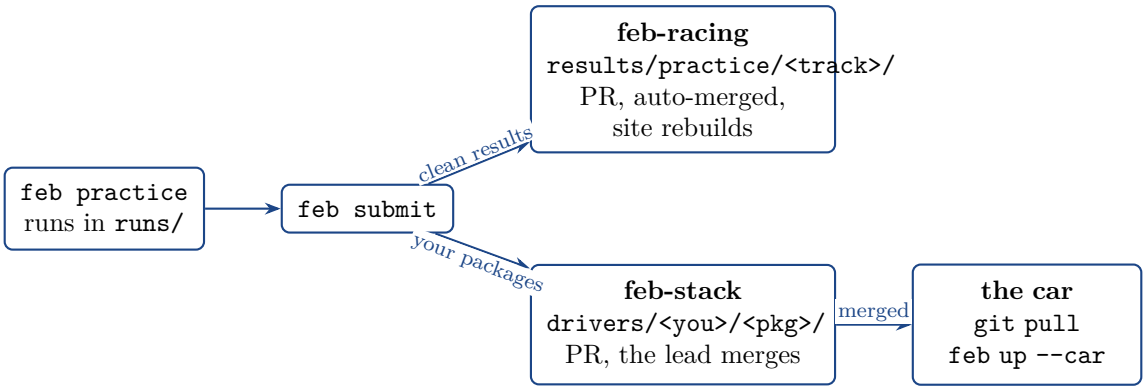
8 Score and submit

```
./feb practice --driver my_driver        # every practice track of its kind, competition
    rules; --headless to hide the window
./feb submit --dry-run                   # what would be posted
./feb submit
```

`practice` runs each track in its own clean container the way the competition does: a warm-up lap and ten timed laps, 300 s limit, 10 s per collision and disqualified above ten, at the 10 Hz scoring rate. The simulator window shows unless you add `--headless`. Each run lands in `runs/<id>/` with a result file, a bag and the logs.

Both commands check that your copy of `feb-racing` is current: `practice` prints a line when it is behind, `submit` refuses until you run `./feb update` (the board reruns every result on the current harness, so a stale one may not verify); `--allow-stale` overrides. Offline, nothing is checked.

`submit` does two things from one command. It posts every clean run (finished, no collisions) not yet posted to the scoreboard as one pull request to `feb-racing`, which checks and merges itself; the site rebuilds and your time appears, unverified until the nightly rerun. And it copies your driver's packages, the `packages:` list in its card, into the team repo `feb-stack` under `drivers/<your name>/`, on a branch named after you, with a pull request to main that the lead reviews. Nothing changed since last time: nothing is pushed. `--no-code` and `--no-results` pick one half.



The team repo. `FEBAutonomous/feb-stack` holds the drivers that drive, one folder per person under `drivers/`, and one copy of the shared packages under `shared/` (`feb_mission`, `feb_cone_ordering`). Nobody edits it by hand. To run a teammate's merged driver in your simulator: `./feb up --ws ~/FEB/feb-stack --driver theirs`.

Part III
The car

9 From a merged driver to the wheels

The Jetson on the car has the same box, the same `~/FEB` folder, and the pad's dongle. Nothing on a laptop or on GitHub is needed to drive; the laptop is only a terminal.

Start-up. Gamepad on *first*, then battery in, SW1 on, a minute for the Jetson to boot. Then a terminal on it, whichever fits:

Where	How
no internet (track, corridor)	45 s after boot the car is Wi-Fi network `feb-car`, password `febautonomous`; join it, then `ssh feb@10.42.0.1`
a known Wi-Fi around	the car joins it; `ssh feb@feb-car` from any laptop on the team tailnet, or its address on that network
a USB-C cable to the Jetson	`ssh feb@192.168.55.1`

Password `feb`. Then:

```
cd ~/FEB/feb-stack && git pull
cd ~/FEB/feb-racing
./feb up --car --driver my_driver --mode-name inspection
```

Wait for *Connected to VESC* and *Opened joystick* in the output. If the pad came on after that, Ctrl-C and run it again: the joystick node only looks once. Ctrl-C ends the run, and so does losing the ssh connection, which is the safe way round; for a long demo put `nohup` in front and `&` at the end, the bumper still stops the car.

On the car the workspace is `feb-stack` by default, so every merged driver is built and the named one launches. A driver that is nowhere in the workspace is looked for in the starter kit next, so `./feb up --car --driver feb_driver --mode-name inspection` works on a car with no merged drivers yet; `--ws FOLDER` points at any other workspace. Either way the launch is the same: the VESC, lidar and gamepad drivers instead of the simulator, the supervisor with the dead-man required and the mode's cap, then your launch file with your car yaml on top. LB held, a human drives; RB held, your code drives; let go, it stops. Foxglove on a laptop at `ws://<the car>:8765` shows the same panels as in the simulator. First drive in `inspection`, then `trackdrive`. The lead's own notes on the car, wiring to first drive, are `docs/build-car.md`.

10 When something is wrong

Setting up:

Symptom	Fix
`Cannot connect to the Docker daemon`	Docker Desktop is not running; open it, wait for the whale icon to settle
the car stops with `STOP: no lidar` every half minute (Windows)	the box's clock is jumping: a second WSL Ubuntu, or Windows itself, keeps resetting it. Windows Settings, Time, *Set time automatically* on; in every other WSL Ubuntu `sudo systemctl disable --now systemd-timesyncd`; then `wsl --shutdown` and start Docker Desktop again
Docker Desktop will not start (Windows)	`wsl --update` in PowerShell, reboot, open Docker Desktop again: an old WSL stops its engine
`this clone sits directly in /home/...`	the clone needs a folder of its own: `mkdir ~/FEB && mv ~/feb-racing ~/FEB/`
`unauthorized` while pulling the image	a stale `ghcr.io` login: `docker logout ghcr.io`, run setup again
`refusing to merge unrelated histories`	you pulled in the old clone; delete it and clone again (Setup)

Running:

Symptom	Fix
`already running` on `./feb up`	a previous run is still going, its terminal was closed: `./feb stop`
`no card.yaml with name '...'`	no card with that `name:` in `stack/` or the starter kit: `./feb new NAME`, section 5
`ros2: command not found`	you are in a laptop terminal: `./feb shell`
my node is not there	it did not build: `./feb logs -n 100`, or `colcon build` in the workspace and read it
the simulator says not connected	the house nodes are still starting; `./feb logs -f`
`port is already allocated`	another box holds the port: `./feb reset`
`mounts denied` (Mac)	`~/FEB` is outside `/Users`: move it
the browser desktop will not connect	reload; still nothing, `./feb reset`
nothing is clean enough to submit	a run with a collision is not posted by default; drive slower, or `--all`
anything weird	`./feb reset`: thirty seconds, nothing of yours is in there

A Every track

`./feb up --track FOLDER` and `./feb practice --track FOLDER` take the folder name; every track is in `tracks/`, one folder each, with its picture. Qualifying tracks are the ones `feb practice` runs by default and the board scores: `loop` and `porto` for walls drivers, `loop_cones` for cones drivers. `--track all` runs the qualifying tracks of every kind; any other track is run by name, one

at a time.

Folder	Kind	Name	Direction
FEB, ours, walls			
`circuit`	walls	FEB Circuit	ccw
`esses`	walls	FEB Esses	ccw
`loop` **`(qualifying)`**	walls	FEB Loop	ccw
FEB, cone edition			
`circuit_cones`	cones	FEB Circuit (cones)	cw
`esses_cones`	cones	FEB Esses (cones)	cw
`loop_cones` **`(qualifying)`**	cones	FEB Loop (cones)	cw
RoboRacer league, walls			
`porto` **`(qualifying)`**	walls	Porto (legacy RoboRacer)	ccw
`rr_berlin`	walls	Berlin	ccw
`rr_vegas`	walls	Vegas	ccw
F1 circuits, walls			
`interlagos`	walls	Interlagos (1:10)	ccw
`monza`	walls	Monza (1:10)	ccw
`silverstone`	walls	Silverstone (1:10)	ccw
`spa`	walls	Spa-Francorchamps (1:10)	ccw
`spielberg`	walls	Spielberg (1:10)	ccw
F1 circuits, cones			
`interlagos_cones`	cones	Interlagos (1:10, cones)	cw
`monza_cones`	cones	Monza (1:10, cones)	cw
`silverstone_cones`	cones	Silverstone (1:10, cones)	cw
`spa_cones`	cones	Spa-Francorchamps (1:10, cones)	cw
`spielberg_cones`	cones	Spielberg (1:10, cones)	cw
Formula Student layouts, cones			
`fs_boa_constrictor`	cones	FS boa constrictor (EUFS)	ccw
`fs_bone`	cones	FS bone (EUFS)	ccw
`fs_comp_2021`	cones	FS comp 2021 (EUFS)	cw
`fs_hairpins_increasing_difficulty`	cones	FS hairpins (EUFS)	cw
`fs_peanut`	cones	FS peanut (EUFS)	ccw
`fs_rectangle`	cones	FS rectangle (EUFS)	ccw
`fs_small_track`	cones	FS small track (EUFS)	cw
`fsg`	cones	FS Germany	cw
`fsi`	cones	FS Italy	cw